高等電腦視覺

作業#(1)

姓名：陳光倫

學號：115318064

指導老師：黃正民

作業系統：Windows11
程式編輯器：Visual Studio 2026
程式語言： C++
編譯模式： Debug（偵錯模式）
平台：x64
使用的 OpenCV 版本：
圖片格式：24-bit uncompressed BMP

1.1

Figure1. BMP Read/Write

511x288 的圖片數據（一般 C++版本）



511x288 的圖片數據(OpencV 版本)



Padding 的計算方式:

以 511x288 的計算:

" rowBeforePadding = 511 × 3 = 1533
padding = (4 - 1533 % 4) % 4 = 3
actualRowSize = 1533 + 3 = 1536
imageBufferSize = 1536 × 288 = 442368 bytes"

Padding 的原因是 BMP 規定每一列的資料大小必須是 4 bytes 的倍數，Padding 放在每列尾

端，不是有效像素。

Pixel 的資料順序:

BMP 的 24-bit pixel 順序是 B, G, R ，若 BMP header 中的高度為正數，pixel data 的排列方向是：(由下到上、由左到右)，因此讀取檔案中的第一列時，那一列其實是影像最下面的一列。

Disgussion 1.1

Padding 的處理是存在於每列像素末端的補充資料，並非有效像素。它的目的是確保每一列的位元組長度能被 4 整除，因為 Padding 長度會跟著影像寬度動態變化，所以不能寫死為 0 或 3 位元組，若在讀取過程忽略 Padding，下一列的讀取指標就會偏移，導致影像出現斜線、嚴重錯位甚至顏色混亂。

像素色彩順序與存儲方向的特性，不同於常見的 RGB 格式，BMP 預設通常採用 BGR 順序儲存顏色資訊。對於正高度（Positive Height）的 BMP 檔案，其像素資料（Pixel Data）是以由下而上（Bottom-Up）的順序進行排列。因此，程式在載入與繪製影像時，必須適當轉置列順序並進行顏色通道轉換，才能正確還原原始畫面。

1.2

Figure 2 順時針旋轉 270°

511x288 的圖片數據（一般 C++版本）



511x288 的圖片數據（OpencV 版本）



輸入：511x288

輸出：288x511

Disgussion 1.2
使用尺寸為 511x288 的非正方形 BMP 圖片進行測試。經過順時針 270 度旋轉後，輸出影像尺寸變為 288x511，符合旋轉後寬度與高度互換的預期。從輸出圖片的內容與方向可以確認影像是逆時針旋轉 90 度，而不是順時針旋轉 90 度，因此座標轉換公式符合題目要求。使用非正方形圖片測試也比使用正方形圖片更容易確認輸出尺寸是否正確。 需注意 BMP 影像的 bottom-up 儲存方式。如果直接按照檔案中的列順序進行座標轉換，可能使輸出圖片的上下方向錯誤。另外，旋轉後影像的寬度已經改變，因此不能直接沿用輸入影像的 row size 和 padding，而必須根據新的寬度重新計算。若 padding 計算錯誤，輸出圖片可能發生傾斜、錯位或無法正常開啟。此旋轉方法需要走訪原始影像中的所有像素一次，因此時間複雜度為 O (WxH)。程式另外建立一個旋轉後的影像 buffer，所以空間複雜度也是 O (WxH)。實驗結果顯示，程式可以自動處理不同影像尺寸，並正確產生順時針旋轉 270 度的 BMP 輸出結果。

Figure 3. Channel Interchange

511x288 的圖片數據（一般 C++版本）



511x288 的圖片數據（OpencV 版本）

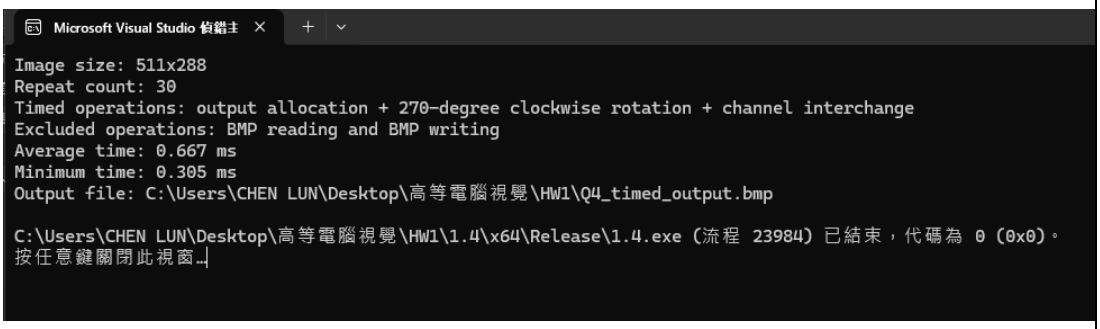


Disgussion 1.3
使用第二題旋轉後的 288×511 BMP 圖片作為輸入。經過通道交換後，輸出圖片的尺寸仍為 288×511，表示此操作只改變每個像素的色彩數值，不會改變影像的寬度、高度或像素位置。從輸出結果可以觀察到圖片的整體色調產生明顯變化，但人物、物體與影像方向仍與旋轉後的圖片相同，符合通道交換的預期結果。 此外 BMP 採用 BGR 排列，在更新通道時必須先保存舊值；若直接依序覆寫 R、G、B，其中一個已修改的數值可能被後續計算再次使用，會導致錯誤。本方法需要走訪影像中的全部像素一次，因此時間複雜度為 O(W×H)。若直接在原影像 buffer 中修改通道，只需要少量暫存變數，因此除了輸入影像本身以外，額外空間複雜度可以視為 O(1)。實驗結果顯示，程式能依照題目指定的規則完成通道交換，且不會改變旋轉後影像的尺寸與幾何內容。

1.4

Figure 4 Execution Time

511x288 的圖片數據（一般 C++版本）



511x288 的圖片數據（OpencV 版本）



Disgussion 1.4

本實驗使用尺寸為 511x288 的 BMP 圖片進行測試，並將旋轉與通道交換重複執行 30 次。實測得到的平均執行時間為__0.667__毫秒，最小執行時間為__0.305__毫秒。(OpencV 版本實測得到的平均執行時間為__0.776__毫秒，最小執行時間為__0.611__毫秒)程式最後產生的輸出圖片尺寸為 288x511，影像方向與第二題旋轉結果相同，色彩則符合第三題指定的通道交換規則。將第四題輸出與依序執行第二題及第三題所得到的結果比較後，兩者內容一致，表示計時程式沒有改變原本影像處理的正確性。

平均執行時間代表多次執行時較具代表性的處理時間，而最小執行時間則較接近系統受到較少背景干擾時的效能。每次測量結果可能略有差異，原因包括作業系統排程、CPU 動態頻率、快取狀態、記憶體配置及其他背景程式。由於 511x288 圖片的資料量不大，單次執行時間可能非常短，因此重複執行並取平均值會比只測量一次更可靠。

BONUS

1.1. bonus

Figure 5. Resize Double Size and One-half Size

放大 2、縮小 1/2 倍　1024*1024、256*256

BMP 讀寫與 padding 計算

```
Original image
  Width: 512
  Height: 512
  Bits per pixel: 24
  Row before padding: 1536 bytes
  Padding per row: 0 bytes
  Actual row size: 1536 bytes
  Image buffer size: 786432 bytes
double resized image
  Width: 1024
  Height: 1024
  Bits per pixel: 24
  Row before padding: 3072 bytes
  Padding per row: 0 bytes
  Actual row size: 3072 bytes
  Image buffer size: 3145728 bytes
  Saved resize, rotation, and channel results for double.
half resized image
  Width: 256
  Height: 256
  Bits per pixel: 24
  Row before padding: 768 bytes
  Padding per row: 0 bytes
```

1024*1024



順時針旋轉 270°

1024*1024



256*256



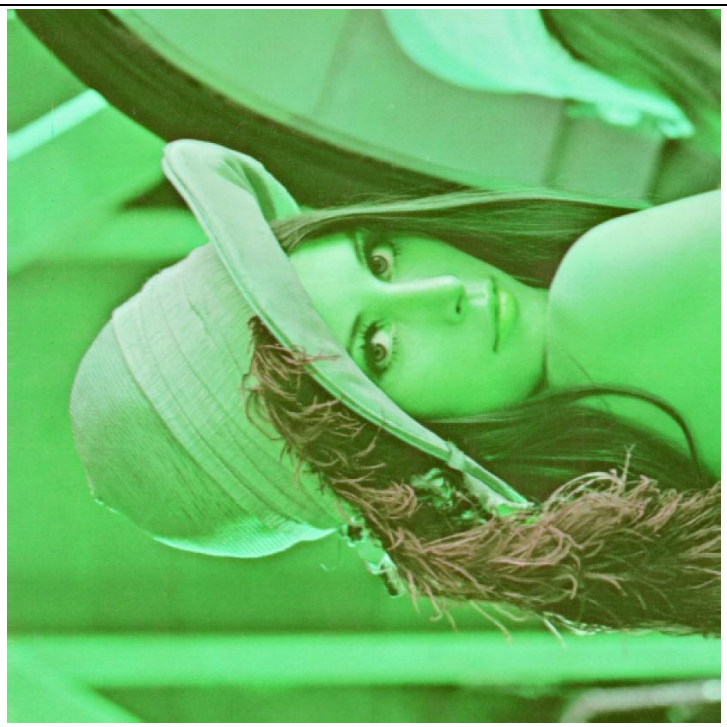
順時針旋轉 270°

256*256



通道交換 R→G、G→B、B→R

1024*1024



256*256



Disgussion 1.1
在影像放大與縮小的實驗中，本程式採用最近鄰插值法決定輸出影像的像素值。此方法會在原始影像中尋找距離輸出位置最近的像素，並直接複製其色彩數值，不會根據周圍像素計算新的顏色。因此，它的計算方式簡單、執行速度快，很適合用來觀察基本的影像取樣效果。 進行影像放大時，輸出影像具有更多像素，但最近鄰插值法不會產生新的影像細節，而是將同一個原始像素複製到多個相鄰位置。以寬度和高度各放大兩倍為例，一個原始像素通常會形成約 2×2 的相同色彩區塊。因此，放大後的圖片雖然尺寸增加，但在斜線、文字及物體輪廓附近可能出現明顯的階梯狀邊緣或方塊感，影像看起來也可能較不平滑。 進行影像縮小時，輸出像素數量減少，最近鄰插值法只會從原圖選取部分像素作為縮小後的結果，其他未被選取的像素資訊則會直接消失。因此，細小紋理、細線及其他高頻細節可能無法保留，規律圖案也可能產生鋸齒、摩爾紋或 aliasing。縮小後雖然仍能保留主要影像內容，但細節通常會比原圖少。 相較之下，雙線性插值法會參考輸出位置附近的四個像素，並依照距離加權計算新的像素值，因此放大後的邊緣通常較平滑，縮小時的視覺變化也較柔和。不過，雙線性插值需要較多計算，執行時間通常高於最近鄰插值。綜合而言，最近鄰插值的優點是實作容易、速度快且能保留原始像素值；缺點則是放大時容易產生方塊感，縮小時容易損失細節。

1.2 bonus

Figure 6 Resize 成 4096x4096

BMP 寫出與 header 資訊

```
C:\Users\CHEN LUN\Desktop\高等電腦視覺\HW1\solution\bonus>.\Bonus2_4096.exe ..\..\WM68d3b04cccdf4.bmp bonus2
4096x4096 resized image
  Width: 4096
  Height: 4096
  Bits per pixel: 24
  Row before padding: 12288 bytes
  Padding per row: 0 bytes
  Actual row size: 12288 bytes
  Image buffer size: 50331648 bytes
Saved all 4096x4096 results.
```

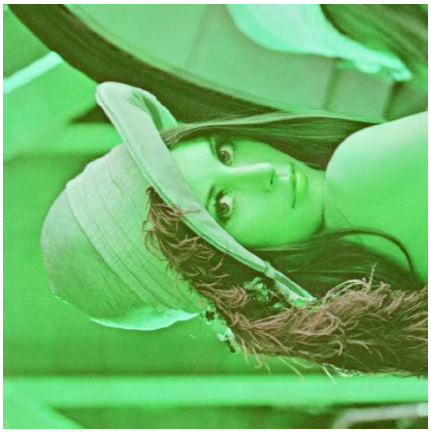
4096*4096



順時針旋轉 270°



Channel interchange



Disgussion 1.2

一張 4096×4096 的 24-bit 影像，其像素資料大小約為 48 MiB，但程式執行時的實際記憶體使用量通常會高於這個數值。原因是 48 MiB 只代表單張影像的 pixel buffer，而影像處理過程中通常需要同時保存多張影像。例如，程式在完成 resize 後仍需要保留 4096×4096 的 resized image，旋轉時又會建立另一個相同大小的 rotated image，因此僅這兩個 pixel buffer 就至少需要約 96 MiB。若原始影像、通道交換結果、檔案讀寫暫存區及其他程式資料也同時存在，總記憶體使用量還會進一步增加。

std::vector 除了儲存像素資料以外，本身也需要保存資料位址、目前元素數量及已配置容量等管理資訊。作業系統與記憶體配置器也可能以固定區塊或記憶體頁面為單位配置空間，因此實際配置量不一定剛好等於程式計算出的像素資料大小。程式執行時所載入的程式碼、函式庫、stack、輸入輸出緩衝區及作業系統管理資訊，也都會占用額外記憶體。因此，工作管理員顯示的記憶體使用量通常會明顯高於單張影像的 48 MiB。

對於這類大型影像資料，heap 比 stack 更適合。Stack 空間通常較小，若將 4096×4096 影

像宣告為大型區域陣列，可能因超過 stack 容量而發生 stack overflow。Heap 的可用空間通常較大，也能在程式執行期間依照實際影像尺寸動態配置。程式使用 std::vector 在 heap 中管理像素資料，不僅能依需求配置大型 buffer，也會在物件生命週期結束時自動釋放記憶體。相較於手動使用 malloc() 和 free()，std::vector 可以降低忘記釋放記憶體、重複釋放或錯誤計算配置大小所造成的 memory leak 與存取錯誤風險。

1.3. bonus

Figure7 不同尺寸的執行時間比較

512*512

```
Timed operations: output allocation + 270-degree clockwise rotation + channel interchange
Excluded operations: resize, BMP reading, BMP writing, CSV output, console output
------------------------------------------------------------
Processing...
Average time: 0.651 ms
Minimum time: 0.599 ms
Time per megapixel: 2.482 ms
Relative pixel count: 1.000 x
Relative execution time: 1.000 x
```

1024*1024

```
Timed operations: output allocation + 270-degree clockwise rotation + channel interchange
Excluded operations: resize, BMP reading, BMP writing, CSV output, console output
------------------------------------------------------------
Processing...
Average time: 4.371 ms
Minimum time: 3.551 ms
Time per megapixel: 4.169 ms
Relative pixel count: 4.000 x
Relative execution time: 6.717 x
```

2048*2048

```
Timed operations: output allocation + 270-degree clockwise rotation + channel interchange
Excluded operations: resize, BMP reading, BMP writing, CSV output, console output
------------------------------------------------------------
Processing...
Average time: 17.872 ms
Minimum time: 16.820 ms
Time per megapixel: 4.261 ms
Relative pixel count: 16.000 x
Relative execution time: 27.463 x
```

4096*4096

```
Timed operations: output allocation + 270-degree clockwise rotation + channel interchange
Excluded operations: resize, BMP reading, BMP writing, CSV output, console output
------------------------------------------------------------
Processing...
Average time: 145.136 ms
Minimum time: 135.707 ms
Time per megapixel: 8.651 ms
Relative pixel count: 64.000 x
Relative execution time: 223.022 x
```

Disgussion 1.3
比較 512x512、1024x1024、2048x2048 與 4096x4096 四種影像尺寸後，可以發現影像尺寸增加時，旋轉與通道交換所需的執行時間也明顯增加。512x512 影像的平均處理時間約為 0.651 ms；當寬度及高度增加為 1024x1024 時，像素數增加為原本的 4 倍，但平均時間增加為約 6.8 倍。2048x2048 的像素數是 512x512 的 16 倍，平均時間則增加為約 33.6 倍；4096x4096 的像素數為原本的 64 倍，但平均時間增加至約 299 倍。由此可看出，實際執行時間雖然會隨像素數增加，但並未完全依照理論像素比例線性成長。 從每百萬像素的處理時間也能觀察到相同現象。512x512 的處理成本約為 2.269 ms/MP，而 1024x1024 與 2048x2048 分別增加至約 3.858 ms/MP 和 4.768 ms/MP；4096x4096 更增加到約 10.603 ms/MP。如果程式效能完全與像素數成正比，各尺寸的 ms/MP 應接近固定值，但實驗結果顯示圖片越大，每個像素的平均處理成本也越高，表示除了像素運算量之外，還存在其他硬體與記憶體因素。 512x512 與 1024x1024 的處理時間較短，而 2048x2048 之後的時間增加幅度開始變大，4096x4096 的效能下降最為明顯。這表示在處理大型影像時，改善演算法迴圈只是其中一部分，若要進一步提升效能，還可以考慮重複利用輸出 buffer、減少中間影像、將旋轉與通道交換合併在同一個迴圈中，以及改善記憶體存取的連續性。

我使用 GPT 和 GEMINI 協助我解釋題目的要求，以及處理我忘記的一些函示庫資料的語法，還有驗證我程式碼錯誤 debug 找不出問題的一些細節。還有報告中文字上的語意修飾盡量不讓我寫得太過於直接，太過於簡潔白話。